

Temporal Analysis of API Call Patterns for Early Detection of Malicious Behaviour

Stuart Asimwe

Department of Computer Science and Software,
Hanyang University, Seoul, South Korea
stuart@hanyang.ac.kr

Author Manuscript - October 2024

Abstract

This paper evaluates how early malicious behaviour can be detected from partial Windows API-call sequences. After exact duplicates and conflicting labels are removed, 13,191 unique traces are divided into fixed training, validation, and test sets. Three representations test the value of call frequency, local order, and longer-range sequence information at prefixes from 5 to 100 calls.

The order-aware model achieves 0.818 balanced accuracy and 0.919 ROC-AUC after 20 calls, reaching 0.904 balanced accuracy after 80 calls. At 20 calls, the LSTM improves macro F1 by 0.051, but it performs significantly worse at 40, 80, and 100 calls. The order-aware sequential policy detects 83.6% of malicious traces by 100 calls with a 5.4% benign false-alarm rate.

On the held-out test set, partial API-call sequences support detection before a complete trace is available. Greater model complexity does not consistently improve detection quality or false-alarm control.

Keywords: malware detection; API-call sequences; temporal analysis; dynamic analysis; behavioural analysis; early detection

1 Introduction

Dynamic malware analysis records how a program interacts with an operating system during execution. Windows API calls capture activities such as process creation, file access, registry modification, memory allocation, and network communication. Because these calls accumulate over time, a practical detector should identify malicious activity before the full trace is available. Previous studies have used API-call sequences to classify Windows malware [2, 5] and to make decisions from partial sequences [4].

Here, “early” refers to the number of calls observed rather than elapsed time. The analysis addresses 3 questions:

- (1) How does detection quality change as the observed prefix grows from 5 to 100 calls?
- (2) Does local call order improve on call-frequency features?
- (3) How early can a sequential policy alert while controlling benign false alarms?

The main contribution is an evaluation of how detection

quality and false alarms change as progressively longer prefixes become available. Exact duplicates and label conflicts are resolved before partitioning, models and thresholds are selected on validation data, and the test partition is reserved for final evaluation. This design addresses known sources of experimental bias in malware research [3].

2 Dataset Audit

The analysis uses the public *Malware Analysis Datasets: API Call Sequences* collection [1]. Each row contains a file hash, a binary label, and the first 100 consecutive non-repeated calls made by the parent process in a Cuckoo Sandbox report. The raw table contains 43,876 rows: 42,797 malware traces and 1,079 benign traces.

For a sequence $x_i = (a_{i,1}, \dots, a_{i,100})$, an exact fingerprint is computed as

$$h_i = \text{SHA256}(\text{LE16}(a_{i,1}) \parallel \dots \parallel \text{LE16}(a_{i,100})), \quad (1)$$

where LE16 encodes each identifier as a signed 16-bit little-endian integer and $\parallel$ denotes byte concatenation. The audit identifies 13,206 distinct fingerprints and 30,670 rows that duplicate another complete sequence. 15 fingerprint groups, covering 1,641 rows, contain both labels. These groups are removed because the available data cannot establish the correct label, and 1 row is retained from each remaining group. This process yields 13,191 unique traces: 12,582 malware and 609 benign.

The cleaned data remain strongly imbalanced. Class weights are computed from the training partition as

$$\omega_c = \frac{N}{2N_c}, \quad (2)$$

where N is the number of training observations and N_c is the number in class c . This avoids synthesising or repeatedly sampling minority observations.

3 Method

3.1 Data Partitions

The deduplicated data are divided into stratified training, validation, and test partitions of 70%, 15%, and 15%,

respectively, using seed 20241022. Deduplication occurs first, so a complete sequence cannot appear in more than 1 partition. The validation partition is used to select regularisation and thresholds, and the test partition is evaluated once after these choices are fixed.

For the call budgets $\mathcal{K} = \{5, 10, 20, 40, 60, 80, 100\}$, each trace produces the prefix

$$x_i^{(k)} = (a_{i,1}, \dots, a_{i,k}), \quad k \in \mathcal{K}. \quad (3)$$

A separate model is fitted at each checkpoint. A result at k therefore measures classification performance using exactly the first k calls.

3.2 Feature Representation

The frequency baseline represents API identifier j by its normalised count:

$$f_{i,j}^{(k)} = \frac{1}{k} \sum_{t=1}^k \mathbb{I}(a_{i,t} = j). \quad (4)$$

This records which calls occurred while discarding their order. The order-aware model uses the feature multiset

$$\phi(x_i^{(k)}) = \{\{a_{i,t}\}_{t=1}^k\} \cup \{\{(a_{i,t}, a_{i,t+1})\}_{t=1}^{k-1}\}, \quad (5)$$

This multiset is mapped to a deterministic 32,768-dimensional sparse vector by feature hashing and then normalised to unit L_2 length. Let $\mathbf{z}_i^{(k)}$ denote the frequency vector in Eq. (4) for the baseline or the hashed vector derived from Eq. (5) for the order-aware model.

Each representation is supplied to a class-weighted, L_2 -regularised logistic regression classifier [6]. At checkpoint k , the predicted probability of malware is

$$\hat{p}_i^{(k)} = \sigma(\mathbf{w}_k^\top \mathbf{z}_i^{(k)} + b_k), \quad \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (6)$$

The tested inverse regularisation strengths are $C \in \{0.1, 1, 10\}$. The value with the highest validation macro F1 is selected, with validation balanced accuracy used to break ties.

The recurrent model maps each API identifier to a 24-dimensional learned embedding and supplies the sequence to a single-layer, unidirectional LSTM with 48 hidden units [7]. Its malware probability is

$$\begin{aligned} \mathbf{e}_t &= \mathbf{E}[a_{i,t}], \\ (\mathbf{h}_t, \mathbf{c}_t) &= \text{LSTM}(\mathbf{e}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}), \\ \hat{p}_i^{(k)} &= \sigma(\mathbf{v}^\top \mathcal{D}_{0.2}(\mathbf{h}_k) + b). \end{aligned} \quad (7)$$

Here, $\mathcal{D}_{0.2}$ is dropout with probability 0.2 during training and the identity function during evaluation. The model uses class-weighted binary cross-entropy, Adam with a learning rate of 0.001, batches of 256, and gradient clipping at 1.0. Training runs for at most 25 epochs in PyTorch 2.5.1 [8]. The validation macro F1 selects the epoch, with balanced accuracy breaking ties, and training stops after 5 epochs without improvement.

3.3 Alert Thresholds

For each checkpoint and candidate false-positive limit $\alpha \in \mathcal{A} = \{0, 0.01, 0.02, 0.05\}$, the validation threshold maximises malware true-positive rate subject to that limit:

$$\tau_{k,\alpha}^* = \arg \max_{\tau \in \mathcal{T}_k} \text{TPR}_{\text{val}}(\tau) \quad \text{s.t.} \quad \text{FPR}_{\text{val}}(\tau) \leq \alpha. \quad (8)$$

where $\mathcal{T}_k$ contains 0, 1, and the validation probabilities at checkpoint k . Ties are resolved by validation macro F1 and then by the higher threshold. Continuous-monitoring candidates require $r \in \{1, 2, 3\}$ consecutive positive checkpoints. For $j \geq r - 1$, the alert indicator at checkpoint k_j is

$$A_i(k_j; r, \alpha) = \prod_{q=0}^{r-1} \mathbb{I}(\hat{p}_i^{(k_{j-q})} \geq \tau_{k_{j-q}, \alpha}^*). \quad (9)$$

An alert occurs when $A_i(k_j; r, \alpha) = 1$. The pair (r^*, α^*) is selected separately for each model to maximise malware coverage while keeping the cumulative validation false-alarm rate at or below 5%. Ties favour the lower cumulative false-alarm rate and then the earlier median alert. The order-aware model selects (3, 0.05), while the LSTM selects (1, 0.02). Both policies are applied unchanged to the test set.

3.4 Metrics

Because 95.4% of cleaned observations are malware, accuracy alone is misleading. Balanced accuracy is

$$\text{BAcc} = \frac{1}{2}(\text{TPR} + \text{TNR}), \quad (10)$$

Macro F1 is the unweighted mean of the malware and benign F1 scores. Reported measures also include ROC-AUC, malware recall, benign specificity, false-positive rate, and precision-recall area for each class. 95% confidence intervals for balanced accuracy, macro F1, and ROC-AUC are estimated from 1,000 stratified bootstrap samples of the fixed test set.

4 Results

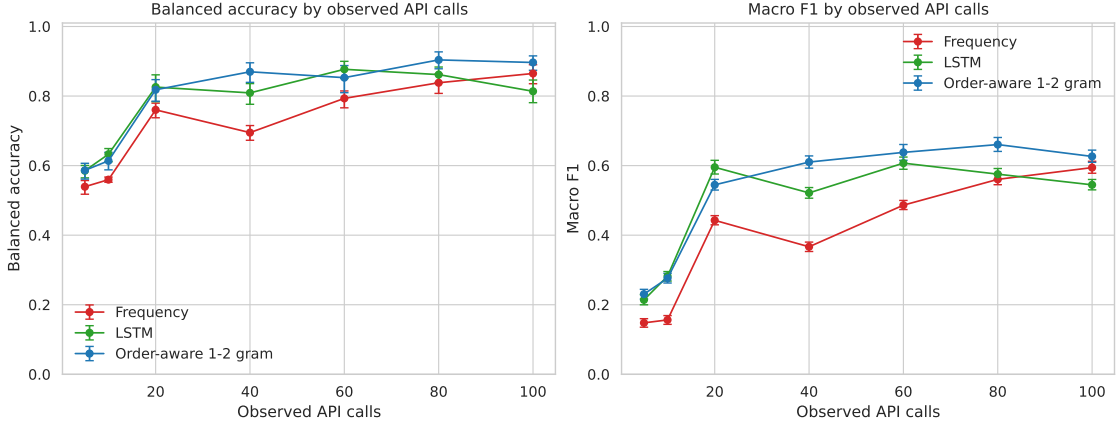
4.1 Prefix Classification

Table 1 reports balanced accuracy and ROC-AUC at each checkpoint. The order-aware representation has higher balanced accuracy than the frequency baseline at all 7 checkpoints. The LSTM has higher balanced accuracy at 10, 20, and 60 calls, while the order-aware model is higher at 5, 40, 80, and 100 calls. The highest values for both reported measures occur with the order-aware model at 80 calls: 0.904 balanced accuracy and 0.965 ROC-AUC.

The paired bootstrap analysis supplements Table 1 with uncertainty for differences between the LSTM and order-aware model. At 20 calls, the LSTM raises macro F1 by 0.051 (95% CI 0.038-0.064), while its 0.008 balanced-accuracy difference includes zero. Its balanced accuracy is lower by 0.061 at 40 calls, 0.042 at 80 calls, and 0.083 at 100 calls; all 3 confidence intervals exclude zero. At 60 calls, the balanced-accuracy

Table 1. Test performance by observed API-call budget. Thresholded measures use validation-selected thresholds.

Calls	Frequency BAcc	Order BAcc	LSTM BAcc	Order AUC	LSTM AUC
5	0.539	0.586	0.585	0.857	0.863
10	0.560	0.614	0.633	0.866	0.800
20	0.760	0.818	0.826	0.919	0.925
40	0.695	0.870	0.809	0.946	0.896
60	0.793	0.853	0.877	0.948	0.939
80	0.838	0.904	0.862	0.965	0.946
100	0.865	0.897	0.814	0.964	0.909

**Figure 1.** Prefix performance. Shaded bands and error bars show 95% stratified bootstrap intervals on the fixed test partition.

difference is uncertain, while the LSTM macro F1 is 0.031 lower (95% CI 0.009-0.053 lower).

The order-aware balanced accuracy rises from 0.586 at 5 calls to 0.870 at 40 calls, falls to 0.853 at 60 calls, peaks at 0.904 at 80 calls, and reaches 0.897 at 100 calls. Each checkpoint uses a separately fitted model and validation-selected threshold. The values therefore compare specific decision points; they do not measure the effect of adding calls to one fixed detector.

4.2 Early-Detection Policy

The validation-selected policy requires 3 consecutive positive checkpoints. On the test set, it detects 16.3% of malware by 20 calls, 71.9% by 60 calls, 80.2% by 80 calls, and 83.6% by 100 calls. Among detected malware, the median alert occurs at 60 calls. 5 of 92 benign test traces trigger an alert, giving a cumulative false-alarm rate of 5.4%.

The LSTM policy selects a single positive checkpoint and detects 86.4% of malware by 100 calls, with a median alert at 20 calls. However, 15 of 92 benign traces trigger an alert, producing a 16.3% test false-alarm rate despite a 4.4% validation rate. The LSTM policy therefore fails to preserve the validation false-alarm constraint on unseen data.

For a call budget k , malware detection coverage is

$$D(k) = \frac{\sum_{i: y_i=1} \mathbb{I}(T_i \leq k)}{\sum_i \mathbb{I}(y_i = 1)}, \quad (11)$$

where T_i is the first checkpoint satisfying Eq. (9). The cumulative benign false-alarm rate is calculated analogously over traces for which $y_i = 0$.

Table 2. Cumulative sequential-policy outcomes.

Calls	Malware detected	Benign false alarms
20	16.3%	2.2%
40	25.4%	4.3%
60	71.9%	4.3%
80	80.2%	4.3%
100	83.6%	5.4%

4.3 Evaluation Constraints

4.3.1 Sequence Integrity

A conventional stratified 80/20 row split of the original data causes 73.7% of test rows to duplicate a sequence in the training set. With this split, the order-aware model reports 0.976 ROC-AUC and 93.4% accuracy. These values exceed the strict deduplicated estimates, but most test rows no longer represent unseen behaviour. This comparison shows why sequence-level deduplication must occur before evaluation.

Exact duplicate removal prevents direct sequence leakage, but near-duplicate variants may remain. Family labels are unavailable, so related malware samples may still occur in different partitions.

4.3.2 Model Constraints

Feature hashing maps unigram and bigram features into 32,768 positions. Different features can be assigned to the same position, causing their counts to be combined and preventing

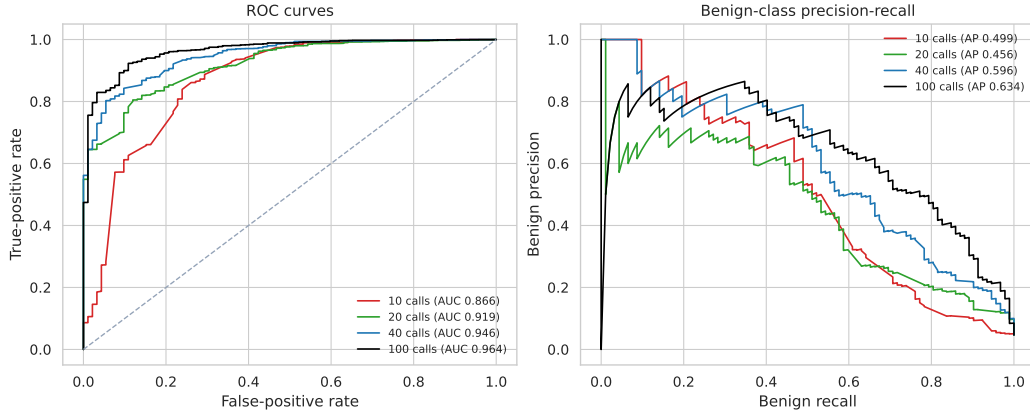


Figure 2. ROC and benign-class precision-recall curves for the order-aware model at selected checkpoints.

the classifier from distinguishing them. The unigram-bigram representation captures only local call order, and the linear classifier cannot model longer dependencies directly. The LSTM can represent longer dependencies, but its additional capacity does not improve performance consistently and its alert thresholds generalise poorly. Threshold selection for both models relies on a limited benign validation sample.

4.3.3 Generalisability

The results are derived from 1 public sandbox dataset and may not transfer directly to other operating systems, sandbox configurations, or deployment environments. Because the dataset lacks collection dates and malware-family labels, chronological and family-held-out evaluation is not possible.

4.3.4 Measurement Boundaries

The source data contain no timestamps, so elapsed detection time cannot be estimated. Call count measures how much of a trace the model has observed. The reported false-alarm rate applies to individual traces in this dataset, rather than to hosts or days in a deployed system.

5 Discussion

Useful behavioural evidence appears early in a trace: the order-aware model reaches 0.818 balanced accuracy after 20 calls. This supports making decisions from partial traces rather than waiting for all 100 calls. Because the dataset records call order without timestamps, the result describes earlier detection by observed calls rather than by elapsed time.

The consistent advantage of local-order features indicates that adjacent call transitions carry information that normalised call counts discard. The recurrent comparison shows that greater sequence capacity alone is insufficient: it provides a narrow improvement at 20 calls but no reliable advantage across the full range of prefixes.

The sequential results expose the practical tradeoff between alert speed and false alarms. Requiring 3 consecutive positive checkpoints delays many alerts, yet it gives substantially better

false-alarm control than the earlier recurrent alerts. This tradeoff matters because even a small per-trace error can create a large review burden when many benign programs are monitored.

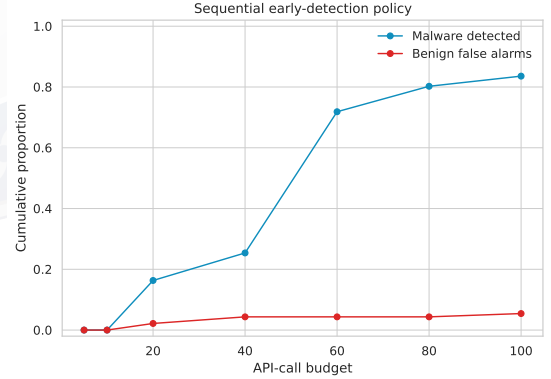


Figure 3. Cumulative malware detection and benign false alarms by call budget.

6 Conclusion

Partial Windows API-call sequences can support malware detection before a complete trace is available. Local call order improves on call frequency at every checkpoint, and the selected sequential policy detects 83.6% of malicious traces by 100 calls with a 5.4% benign false-alarm rate. These results support early detection within this dataset while showing that alert speed must be considered together with false alarms.

In future work, I plan to evaluate chronologically collected traces, hold out complete malware families, and include more benign samples. I also want to work with data that retain repeated calls and timestamps, then compare alternative recurrent and transformer architectures across multiple training seeds using the same deduplication, validation, and held-out test procedure.

Artifacts. The experiment code, configuration, and results are available in the project repository [9].

References

- [1] A. Oliveira, “Malware Analysis Datasets: API Call Sequences,” *IEEE DataPort*, 2019.
- [2] F. O. Catak and A. F. Yazı, “A benchmark API call dataset for Windows PE malware classification,” *arXiv preprint arXiv:1905.01999*, 2019.
- [3] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, “TESSERACT: Eliminating experimental bias in malware classification across space and time,” in *Proc. 28th USENIX Security Symp.*, 2019, pp. 729-746.
- [4] P. Maniriho, A. N. Mahmood, and M. J. M. Chowdhury, “EarlyMalDetect: A novel approach for early Windows malware detection based on sequences of API calls,” *arXiv preprint arXiv:2407.13355*, 2024.
- [5] M. Tang and Q. Qian, “Dynamic API call sequence visualisation for malware classification,” *IET Information Security*, vol. 13, no. 4, pp. 367-377, 2019.
- [6] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [7] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.
- [8] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [9] S. Asiimwe, “API Call Patterns,” GitHub repository, 2024. <https://github.com/stuartasiimwe7/api-call-patterns>